

Module 9: React

1. ReactJS-HOL

1. What is a Single Page Application (SPA)?

A **Single Page Application (SPA)** is a web application that loads only one HTML page initially. When users navigate through the application, only the required content is updated instead of reloading the entire webpage.

Benefits of SPA

- Faster page loading
- Better user experience
- Reduced server requests
- Smooth navigation
- Less bandwidth usage

Examples:

- Gmail
- Facebook
- Netflix
- Google Maps

2. What is React?

React is an open-source JavaScript library developed by Facebook (Meta) for building user interfaces, especially for Single Page Applications.

React uses reusable components to build interactive websites efficiently.

3. How React Works

React follows these steps:

1. User performs an action.
2. React updates the component state.
3. Virtual DOM is updated.
4. React compares the new Virtual DOM with the old Virtual DOM.
5. Only the changed elements are updated in the Real DOM.
6. The browser displays the updated UI.

4. Difference Between SPA and MPA

SPA	MPA
Loads a single HTML page	Loads a new HTML page every request
Faster navigation	Slower navigation
Uses JavaScript heavily	Uses server-side rendering
Better user experience	Traditional navigation
Examples: React, Angular, Vue	Examples: Amazon, Wikipedia

5. Advantages and Disadvantages of SPA

Advantages

- Fast performance
- Better user experience
- Less server load

- Easy API integration
- Efficient caching

Disadvantages

- SEO can be difficult
- Larger initial loading time
- JavaScript dependency
- Browser history management is more complex

6. What is Virtual DOM?

The Virtual DOM is a lightweight copy of the Real DOM.

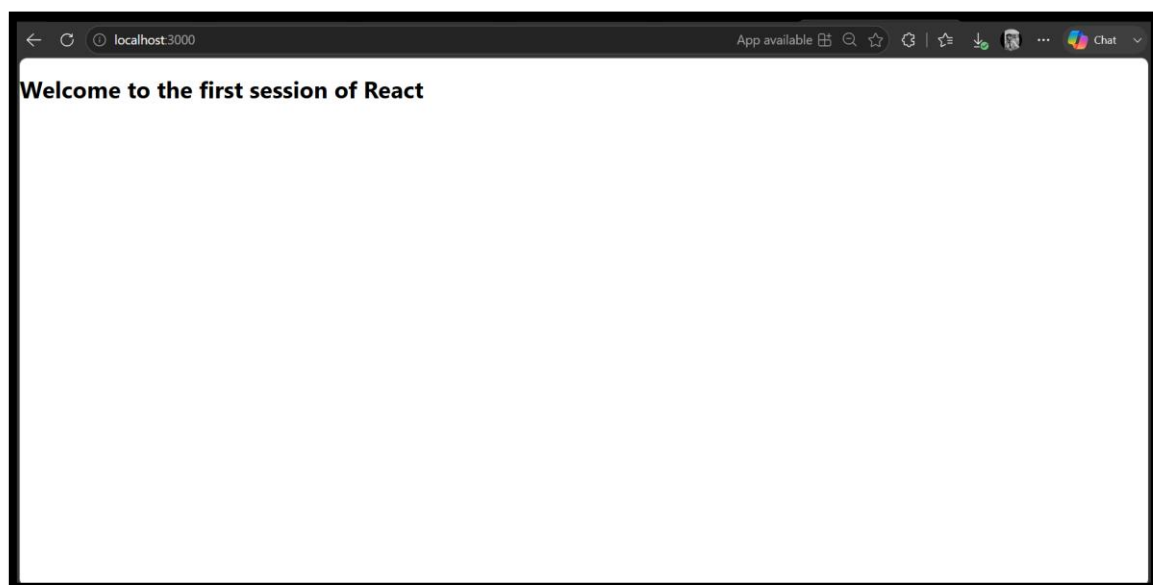
Instead of updating the browser's DOM directly, React:

- Creates a Virtual DOM
- Compares changes (Diffing)
- Updates only the modified elements

This process makes React applications much faster.

7. Features of React

- Component-Based Architecture
- Virtual DOM
- JSX Syntax
- One-way Data Binding
- Reusable Components
- High Performance
- Easy Learning Curve
- Strong Community Support



2. ReactJS-HOL

1. What are React Components?

A **React Component** is an independent and reusable piece of code that returns JSX (HTML-like syntax). Components help divide the user interface into smaller, manageable, and reusable parts.

Examples:

- Header
- Footer
- Home Page
- Navigation Bar
- Contact Page

2. Components vs JavaScript Functions

React Component	JavaScript Function
Returns JSX (HTML elements)	Returns values such as numbers, strings, or objects
Used to build UI	Used for logic and calculations
Starts with a Capital Letter	Can have any valid name
Can maintain state (Class Components)	Cannot maintain React state
Can receive props	Receives normal function parameters

3. Types of React Components

1. Functional Components

- Simple JavaScript functions.
- Return JSX.
- Easier to write and understand.
- Preferred in modern React.

Example:

```
function Home() {  
  return <h1>Home Component</h1>;  
}
```

2. Class Components

- Created using ES6 classes.
- Extend React.Component.
- Include a render() method.
- Can use lifecycle methods and state.

Example:

```
import React, { Component } from "react";  
class Home extends Component {  
  render() {  
    return <h1>Home Component</h1>;  
  }  
}  
export default Home;
```

4. Class Component

A Class Component is a JavaScript class that extends React.Component.

Syntax:

```
class ComponentName extends React.Component {  
  render() {  
    return(JSX);  
  }  
}
```

5. Functional Component

A Functional Component is a simple JavaScript function that returns JSX.

Syntax:

```
function ComponentName() {  
  return(JSX);  
}
```

6. Component Constructor

The constructor initializes the component.

Syntax:

```
constructor(props){  
  super(props);  
}
```

The constructor is used to:

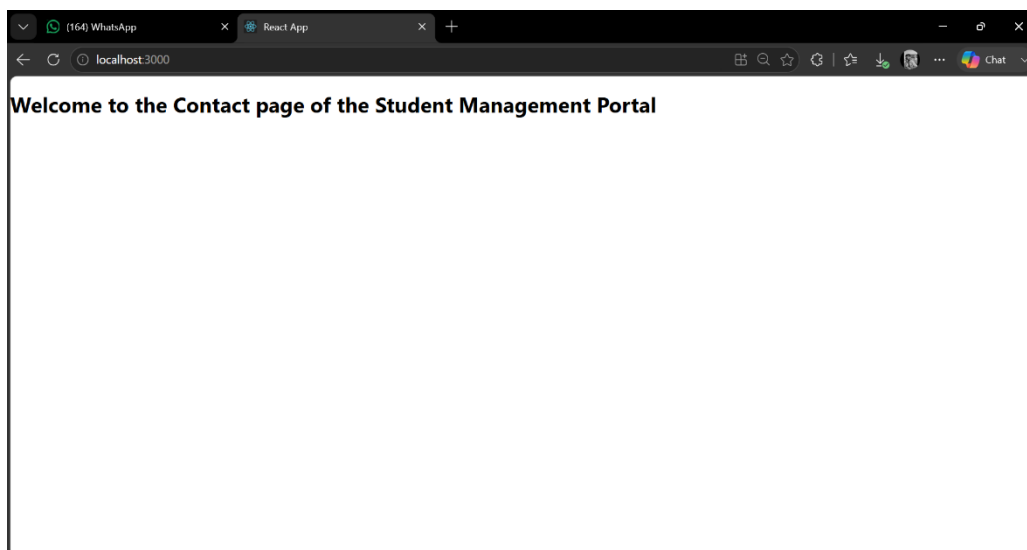
- Initialize state
- Bind event handlers
- Call the parent constructor using super()

7. render() Function

The render() function is mandatory in Class Components. It returns the JSX that should be displayed on the screen.

Example:

```
render() {  
  return(<h1>Hello React</h1>);  
}
```



3. ReactJS-HOL

1. What are React Components?

A **React Component** is a reusable block of code that returns JSX (JavaScript XML). Components make applications modular, reusable, and easier to maintain.

2. Components vs JavaScript Functions

React Component	JavaScript Function
Returns JSX	Returns any JavaScript value
Used for UI	Used for calculations and logic
Starts with a capital letter	Can have any valid function name
Can receive Props	Receives function parameters
Can manage state (using Hooks or Class Components)	Cannot manage React state

3. Types of Components

Functional Component

- Written as a JavaScript function.
- Returns JSX.
- Uses Hooks for state management.
- Preferred in modern React.

Class Component

- Written using ES6 classes.
- Extends React.Component.
- Uses the render() method

4. Class Component

A Class Component extends React.Component and must implement the render() method.

Example:

```
class Home extends React.Component {  
  render() {  
    return <h1>Hello</h1>;  
  }  
}
```

5. Functional Component

A Functional Component is a JavaScript function that returns JSX.

Example:

```
function Home() {  
  return <h1>Hello</h1>;  
}
```

6. Component Constructor

The constructor initializes a Class Component.

Syntax:

```
constructor(props) {
```

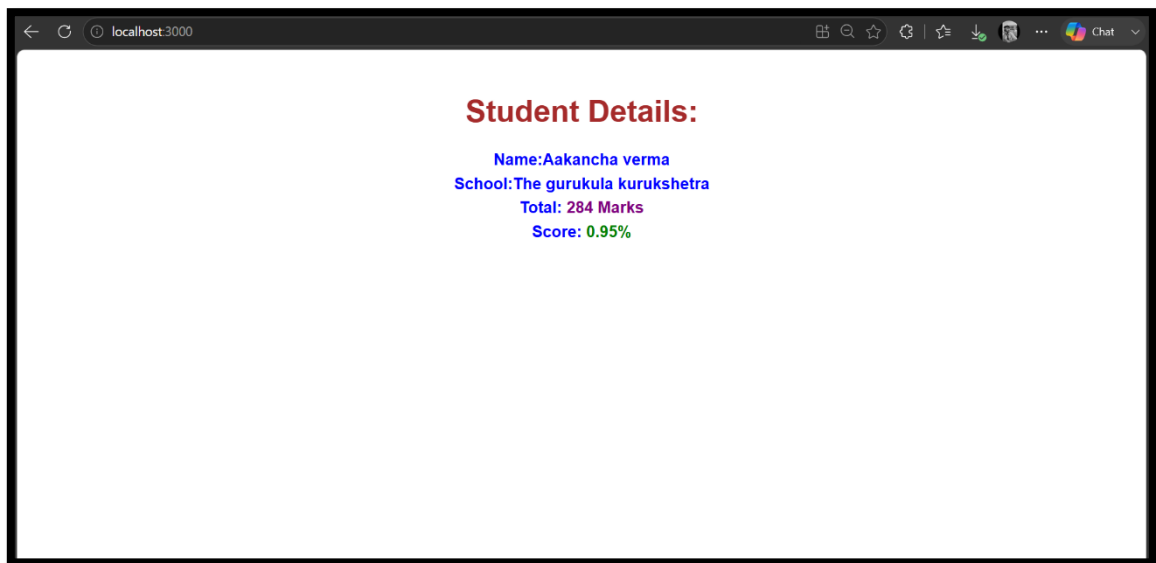
```
    super(props);  
  }
```

7. render() Function

The render() method returns the JSX that should appear on the browser.

Example:

```
render() {  
  return <h1>Welcome</h1>;  
}
```



4. ReactJS-HOL

1. What is the Component Lifecycle?

Every React Class Component goes through different stages during its lifetime. These stages are called the **Component Lifecycle**.

The lifecycle is divided into three phases:

- **Mounting** – Component is created and inserted into the DOM.
- **Updating** – Component is re-rendered when props or state change.
- **Unmounting** – Component is removed from the DOM.

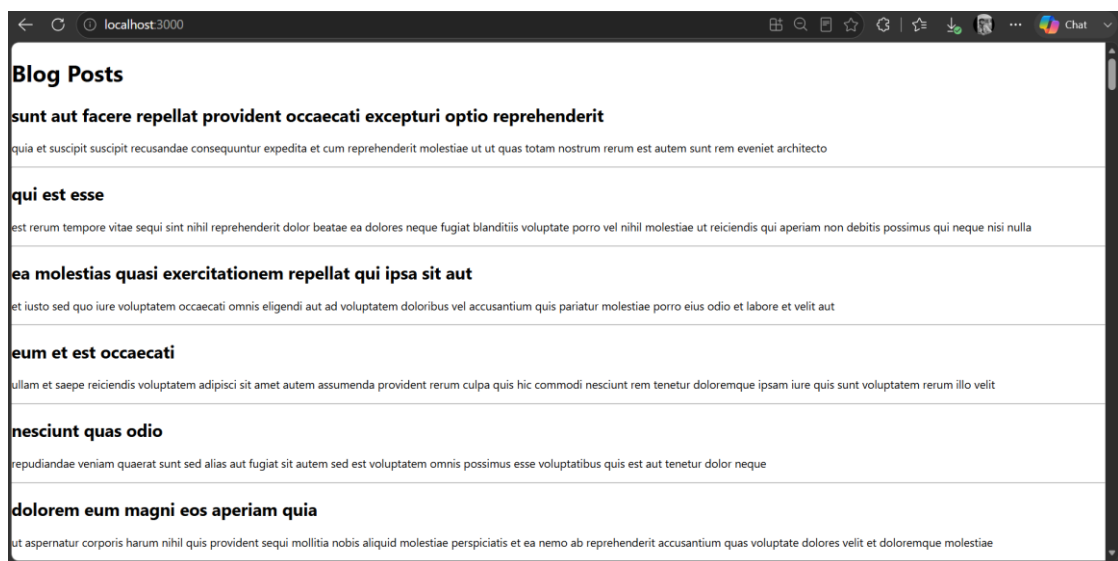
2. Benefits of Lifecycle Methods

- Fetch data from APIs.
- Update component state.
- Handle errors.
- Improve performance.
- Perform cleanup operations.

3. Common Lifecycle Methods

Lifecycle Method	Purpose
constructor()	Initializes state
render()	Displays UI
componentDidMount()	Executes after the component is mounted
componentDidUpdate()	Executes after updates
componentWillUnmount()	Executes before component removal
componentDidCatch()	Handles errors in child components

4. Component Rendering Sequence



5. ReactJS-HOL

← ↻ 🕒 localhost:3000

🏠 🔍 ⭐ ⚙️ | 🗺️ 👤 ⋮ 🗨️ Chat ▾

Cohorts Details

INTADMDF10 - .NET FSD
Started On
22-Feb-2022
Current Status
Scheduled
Coach
Aathma
Trainer
Jojo Jose

ADM21JF014 - Java FSD
Started On
10-Sep-2021
Current Status
Ongoing
Coach
Apoorv
Trainer
Elisa Smith

CDBJF21025 - Java FSD
Started On
24-Dec-2021
Current Status
Ongoing
Coach
Aathma
Trainer
John Doe

INTADMJF12 - Java FSD
Started On
22-Feb-2022
Current Status
Scheduled
Coach
Ibrahim
Trainer
To Be Assigned

CDE22JF011 - Java FSD
Started On
24-Dec-2021
Current Status
Ongoing
Coach
Apoorv
Trainer
Emma Swan

INTADMDF09 - Dataware Housing
Started On
22-Feb-2022
Current Status
Scheduled
Coach
Aathma
Trainer
Babjee Rao

ADM22DF001 - .NET FSD
Started On
10-Sep-2021
Current Status
Ongoing
Coach
Ibrahim
Trainer
Marie Curie

9. ReactJS-HOL

1. Features of ES6

ECMAScript 6 (ES6) introduced several modern JavaScript features:

- let
- const
- Arrow Functions
- Classes
- Template Literals
- Destructuring
- Modules
- Spread Operator (...)
- Rest Parameters
- Promises
- Map and Set

2. Difference between var and let

var	let
Function scoped	Block scoped
Can be redeclared	Cannot be redeclared
Hoisted	Hoisted but cannot be used before declaration
Older syntax	ES6 syntax

Example:

```
let age = 20;
```

3. JavaScript const

Used to declare constants. `const PI = 3.14;` The value cannot be reassigned.

4. Arrow Functions

Traditional Function

```
function add(a,b){  
  return a+b;  
}
```

Arrow Function

```
const add=(a,b)=>a+b;
```

5. Destructuring

Extract values from arrays.

```
const arr=[1,2,3];  
const [a,b,c]=arr;
```

6. Spread Operator

Merge arrays.

```
const c=[...a,...b];
```

7. Map()

Used to iterate through arrays.

```
players.map(player=>player.name)
```

The screenshot shows a web browser at localhost:3000. The page has a title 'List of Players' and displays a table of players with their names and scores. Below the table, there is a section titled 'Players with Score below 70' which lists the players whose scores are less than 70.

Name	Score
Virat Kohli	95
Rohit Sharma	85
Shubman Gill	75
KL Rahul	60
Hardik Pandya	55
Ravindra Jadeja	72
Rishabh Pant	68
Surya Kumar	90
Bumrah	45
Mohammed Siraj	65
Kuldeep Yadav	78

Players with Score below 70

- KL Rahul - 60
- Hardik Pandya - 55
- Rishabh Pant - 68
- Bumrah - 45
- Mohammed Siraj - 65

10. ReactJS-HOL

1. What is JSX?

JSX (JavaScript XML) is a syntax extension for JavaScript that allows developers to write HTML-like code inside JavaScript.

Example:

```
const element = <h1>Welcome to React</h1>;
```

JSX is converted into `React.createElement()` calls by Babel.

2. What is ECMAScript (ES6)?

ECMAScript (ES6) is the standard version of JavaScript that introduced features such as:

- `let` and `const`
- Arrow Functions
- Classes
- Template Literals
- Destructuring
- Modules
- Spread Operator

3. `React.createElement()`

JSX:

```
<h1>Hello React</h1>
```

is internally converted into:

```
React.createElement("h1", null, "Hello React");
```

4. Rendering JSX to DOM

React renders JSX into the browser using:

```
ReactDOM.render(<App />, document.getElementById("root"));
```

(For newer React versions, `createRoot()` is used automatically in the default project.)

5. JavaScript Expressions in JSX

Use curly braces `{ }` to include JavaScript expressions.

Example:

```
const name = "Aakancha";
```

```
<h2>Welcome {name}</h2>
```

6. Inline CSS in JSX

Example:

```
<h2 style={{ color: "red" }}>
```

Office Rent

```
</h2>
```

Notice:

- CSS properties are written in **camelCase**
- Styles are written inside **double curly braces**

← ↻ 🌐 localhost:3000

🏠 🔍 ⭐ 🔄 ⚙️ ⬇️ 🖨️ ⋮ 🗯 Chat ▾

Office Space Rental App

Featured Office



Name: Regus Business Center

Rent: ₹55000

Address: Salt Lake, Kolkata

Available Office Spaces



← ↻ 🌐 localhost:3000

🏠 🔍 ⭐ 🔄 ⚙️ ⬇️ 🖨️ ⋮ 🗯 Chat ▾

Regus Business Center

Rent: ₹55000

Salt Lake, Kolkata



WeWork Office

Rent: ₹75000

Bangalore



Smart Workspace

Rent: ₹45000

Hyderabad

11. ReactJS-HOL

1. What are React Events?

React Events are actions triggered by the user, such as:

- Mouse Click
- Mouse Hover
- Key Press
- Form Submit
- Change Even

React handles these events using **Event Handlers**.

Example:

```
<button onClick={handleClick}>Click Me</button>
```

2. Event Handler

An Event Handler is a JavaScript function that executes when an event occurs.

Example:

```
function handleClick() {  
  alert("Button Clicked");  
}
```

3. Synthetic Event

React wraps browser events inside **Synthetic Events**.

Benefits:

- Cross-browser compatibility
- Better performance
- Same behavior across browsers

Example:

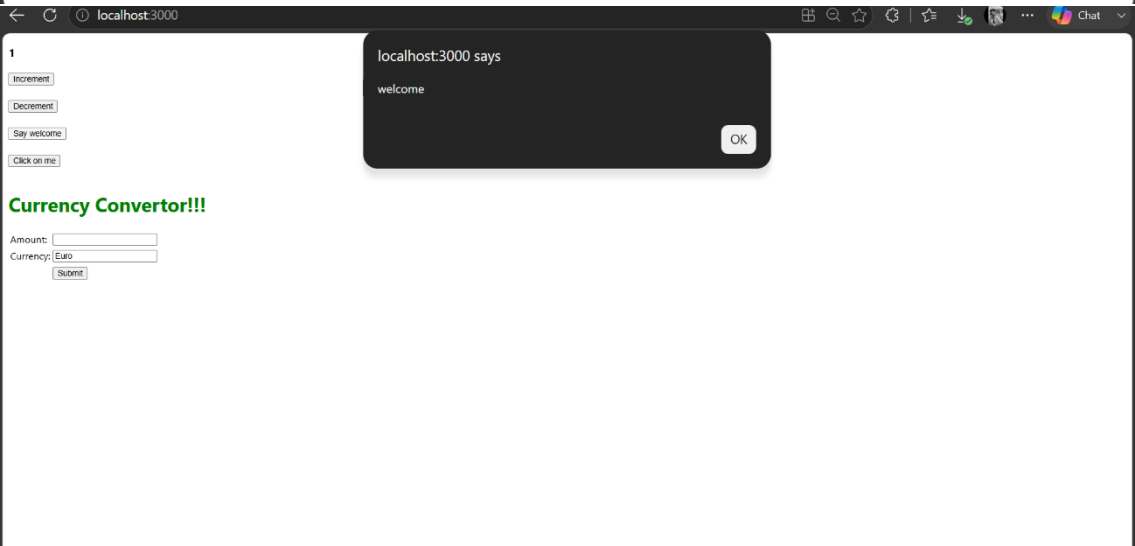
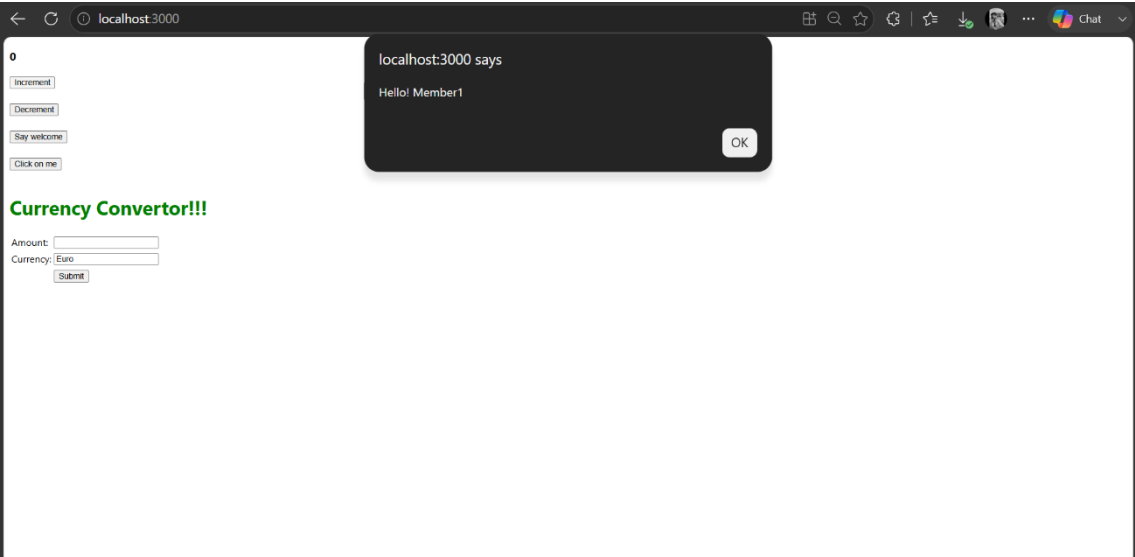
```
<button onClick={handleClick}>
```

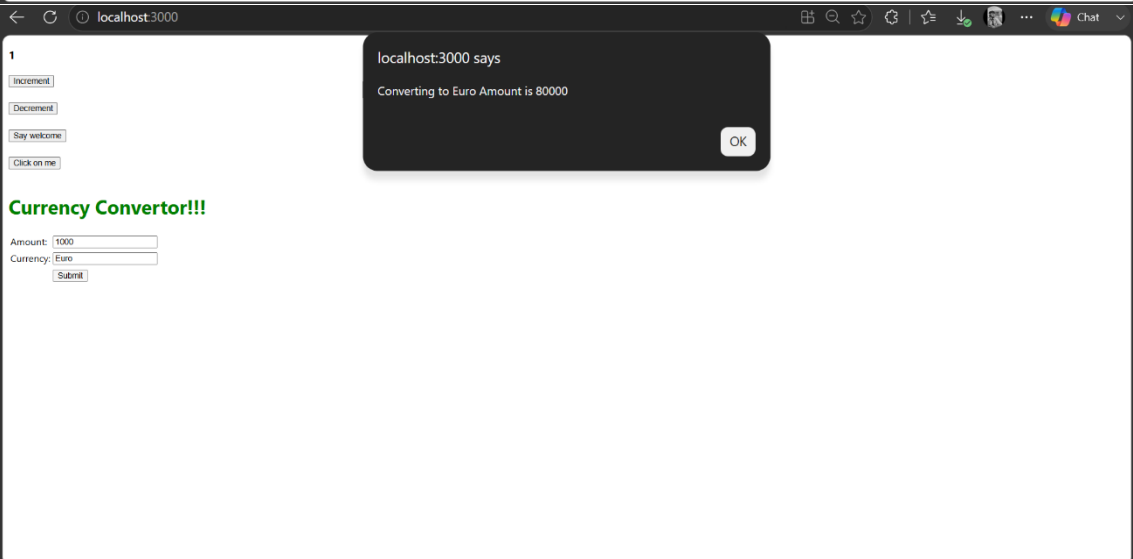
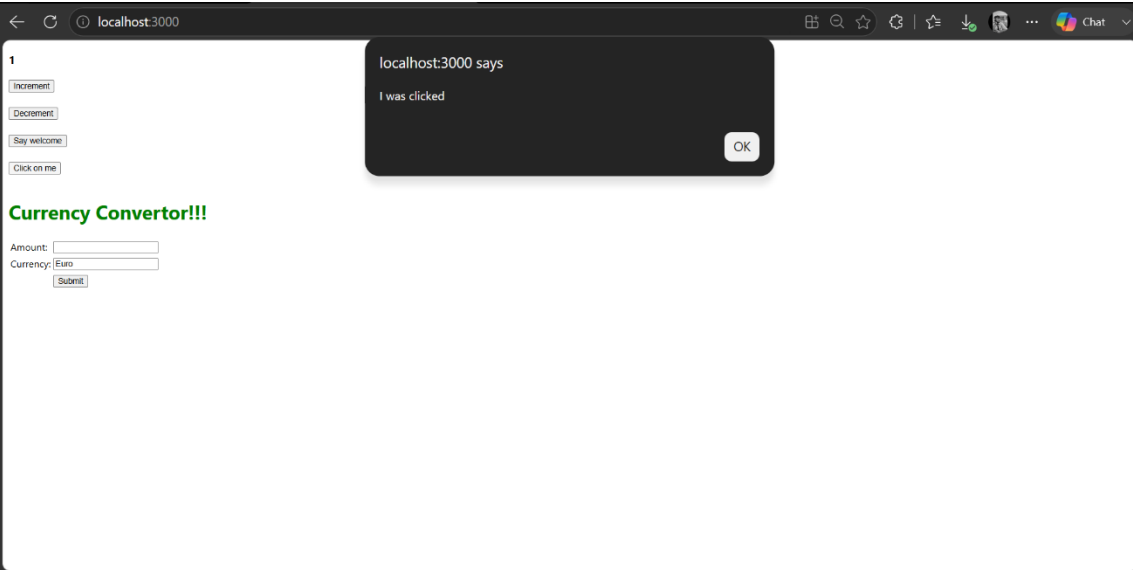
4. React Event Naming Convention

React uses **camelCase** for event names.

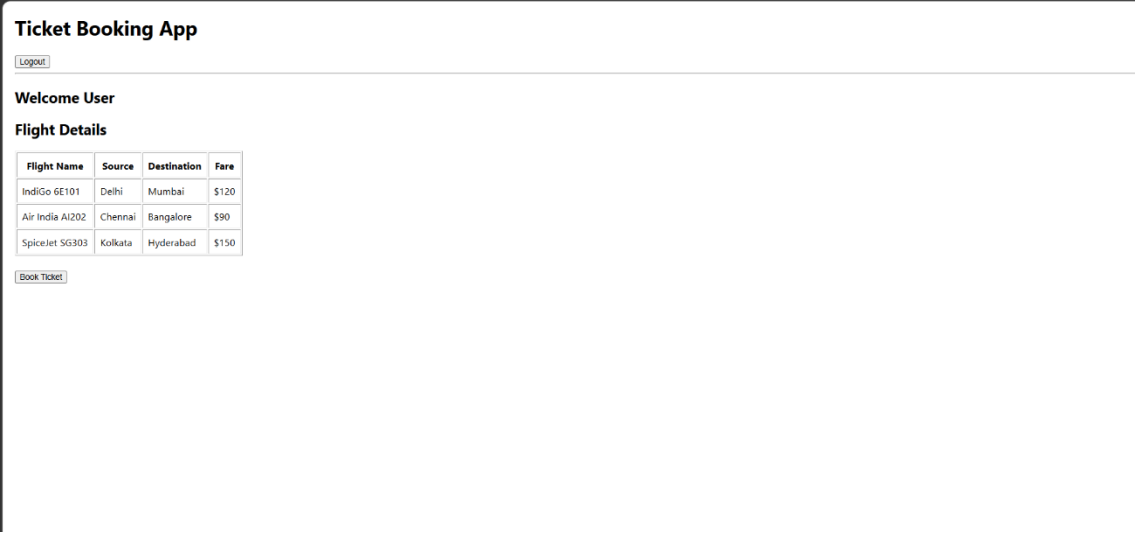
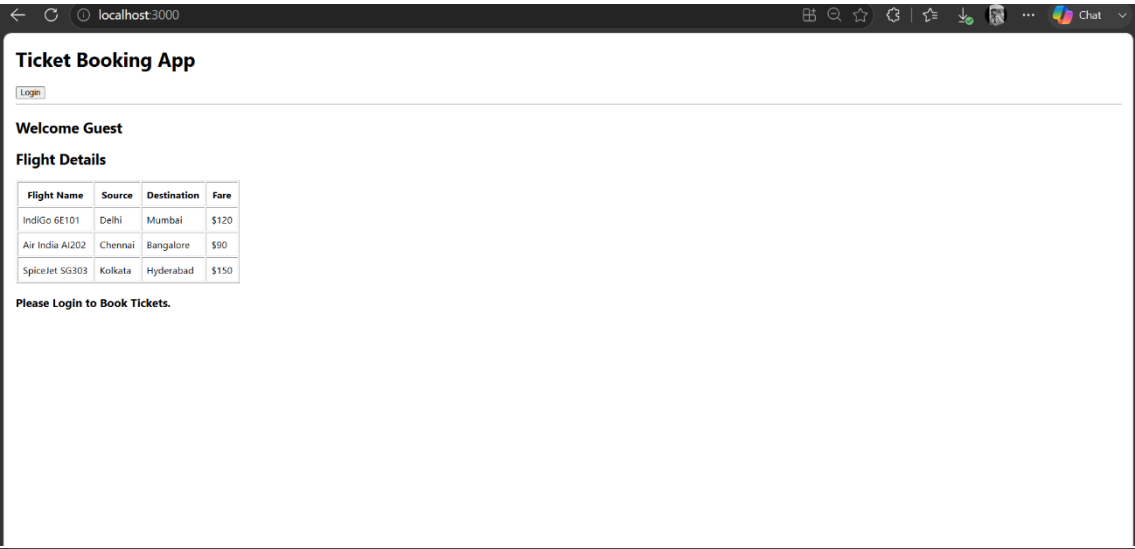
Examples:

HTML	React
onclick	onClick
onchange	onChange
onsubmit	onSubmit
ondblclick	onDoubleClick
onmouseover	onMouseOver





12. ReactJS-HOL



13. ReactJS-HOL

1. Explain Various Ways of Conditional Rendering in React

Conditional rendering means **displaying different UI elements or components based on a condition.**

Methods of Conditional Rendering

a) Using if Statement

Used when multiple conditions need to be checked.

```
function App() {  
  const isLoggedIn = true;  
  if (isLoggedIn) {  
    return <h1>Welcome User</h1>;  
  }  
  return <h1>Please Login</h1>;  
}
```

b) Using Ternary Operator (?:)

Best for simple if-else conditions.

```
{ isLoggedIn ? <Home /> : <Login /> }
```

c) Using Logical AND (&&)

Displays a component only if the condition is true.

```
{ isAdmin && <AdminPanel /> }
```

d) Using Element Variables

Store JSX inside a variable.

```
let message;  
if(isLoggedIn)  
  message = <Home />;  
else  
  message = <Login />;  
return (<div>{message}</div>);
```

e) Using Switch Statement

Useful when there are multiple possible conditions.

```
switch(role){  
  case "Admin":  
    return <Admin />;  
  case "User":  
    return <User />;  
  default:  
    return <Guest />;  
}
```

2. Explain How to Render Multiple Components

React applications are built from multiple reusable components. A parent component can render several child components.

Example:

```
import BookDetails from "./BookDetails";  
import BlogDetails from "./BlogDetails";  
import CourseDetails from "./CourseDetails";  
function App() {  
  return (  
    <div>  
      <BookDetails />  
      <BlogDetails />  
      <CourseDetails />  
    </div>  
  );  
}
```

Advantages

- Improves code reusability.
- Makes applications easier to maintain.
- Keeps code modular and organized.
- Simplifies debugging

3. Define List Component

A list component is a React component that displays a collection of items by iterating over an array.

Example:

```
const books = ["React", "Angular", "MongoDB"];
function BookList() {
  return (
    <ul>
      {books.map((book, index) =>
        <li key={index}>{book}</li>
      )}
    </ul>
  );
}
```

Benefits

- Displays dynamic data.
- Reduces repetitive code.
- Works efficiently with arrays.

4. Explain Keys in React Applications

A **key** is a unique identifier assigned to elements in a list. React uses keys to identify which items have changed, been added, or been removed.

Example:

```
books.map((book) =>
  <li key={book.id}>
    {book.name}
  </li>
)
```

Rules

- Keys should be unique among siblings.
- Prefer IDs from your data.
- Avoid using array indexes if the list order can change.

Advantages

- Improves rendering performance.
- Helps React update only the changed items.
- Prevents unnecessary re-rendering.

5. Explain How to Extract Components with Keys

When rendering lists, it's common to extract the repeated JSX into a separate component. The **key should be assigned where the list is created**, not inside the extracted component.

Example:

Book.js

```
function Book(props){
  return <li>{props.name}</li>;
}
```

export default Book;

BookList.js

```
import Book from "./Book";
const books = [
  {id:1,name:"React"},
  {id:2,name:"Angular"}
];
function BookList(){
  return(
    <ul>
      {
        books.map(book =>
          <Book
            key={book.id}
            name={book.name}
          />
        )
      }
    </ul>
  );
}
```

export default BookList;

Why?

The key belongs on the <Book /> component in the map() call because React uses it to track list items while reconciling the UI.

6. Explain React Map and map() Function

The map() function is a JavaScript array method used in React to transform an array of data into an array of JSX elements.

Syntax

```
array.map((item,index)=>{
  return JSX;
});
```

Example

```
const fruits = ["Apple","Orange","Banana"];
function FruitList(){
  return(
    <ul>
      {
        fruits.map((fruit,index)=>
          <li key={index}>
            {fruit}
          </li>
        )
      }
    </ul>
  );
}
```

Advantages

- Converts arrays into UI elements.
- Avoids repetitive code.

- Makes rendering dynamic data simple.
- Commonly used with APIs and databases.

Blogger App	
Book Details	
Book Name	Price
Master React	670
Deep Dive into Angular 11	800
Mongo Essentials	450

Blog Details	
React Learning	
Stephen Biz	
Welcome to learning React!	
Installation	
Schweider	
You can install React from npm.	

Course Details	
Course	Date
Angular	4/5/2021
React	6/5/2021

Blogger App		
Course Details	Book Details	Blog Details
Angular	Master React	React Learning
4/5/2021	670	Stephen Biz
React	Deep Dive into Angular 11	Welcome to learning React!
6/5/2021	800	Installation
	Mongo Essentials	Schweider
	450	You can install React from npm.